Transformer

Część pierwsza ćwiczeń z transformers

Po co nowy mechanizm?

- W tradycyjnych RNN/LSTM informacja przepływa sekwencyjnie. Mechanizmy uwagi natomiast pozwalają modelowi skupić się na róż wejścia przy generowaniu każdego elementu wyjścia.
- W Transformerach uwaga jest podstawowym mechanizmem: każdy token oblicza "ważność" innych tokenów przez iloczyn skalarny sw (queries) z kluczami (keys) i wykorzystuje wartości (values) do zbudowania reprezentacji.

Attention is All You Need

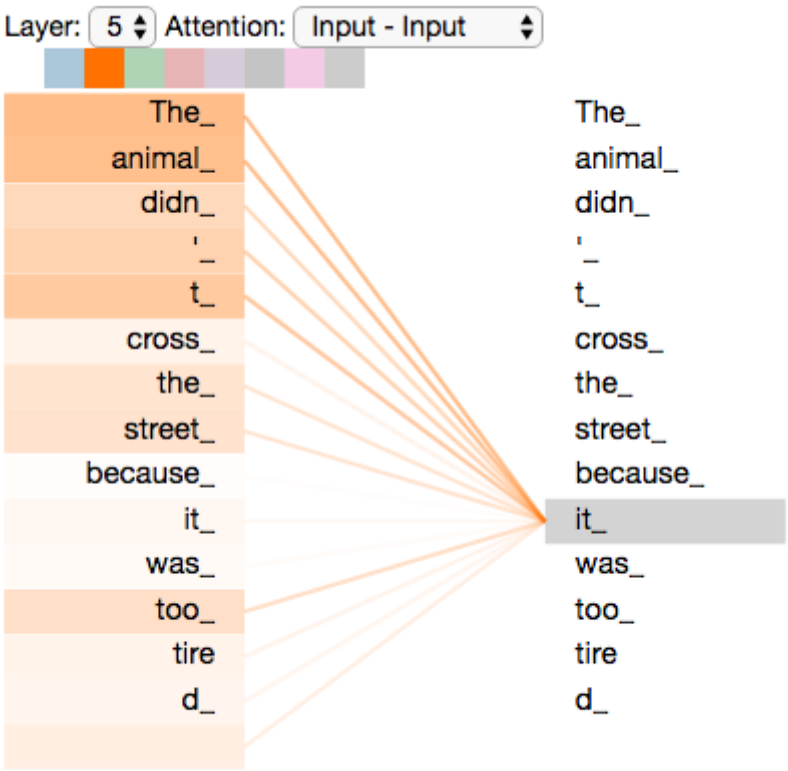
Mechanizm, który odpowiada za dobrą reputację transformerów to mechanizm attencji / uwagi. Mechanizm ten został zaprezentowany w r Attention is All You Need

1 Wejście

Na wejściu mamy takie zdanie:

```
"The animal didn't cross the street because it was too tired"
```

W tym zdaniu modele językowe moga mieć problem zrozumieć do czego odnosi się słowo 'it".



2 Embeddingi

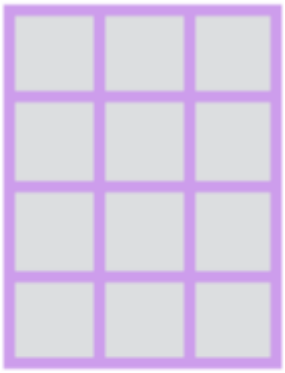
Przyszły do nas dwa słowa, które mają swoje odpowiadające embeddingi - x 1 i x 2



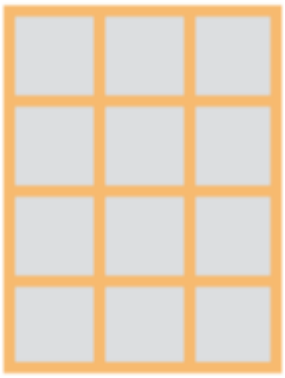
3 Query, key i value

- Q: query / zapytania — reprezentacja tego, czego "szukamy".
- K: key / klucze — reprezentacje elementów, do których porównujemy zapytania.
- V: values / wartości — zawierają informacje, które będziemy agregować (przez ważoną sumę).

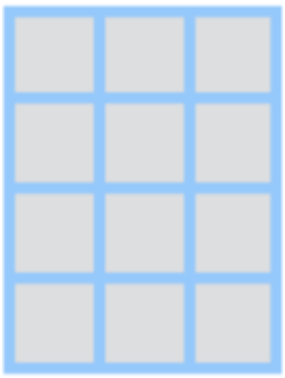
Podczas treningu modelu tworzymy 3 macierze - wagi dla wartości query, key i value:



W^Q

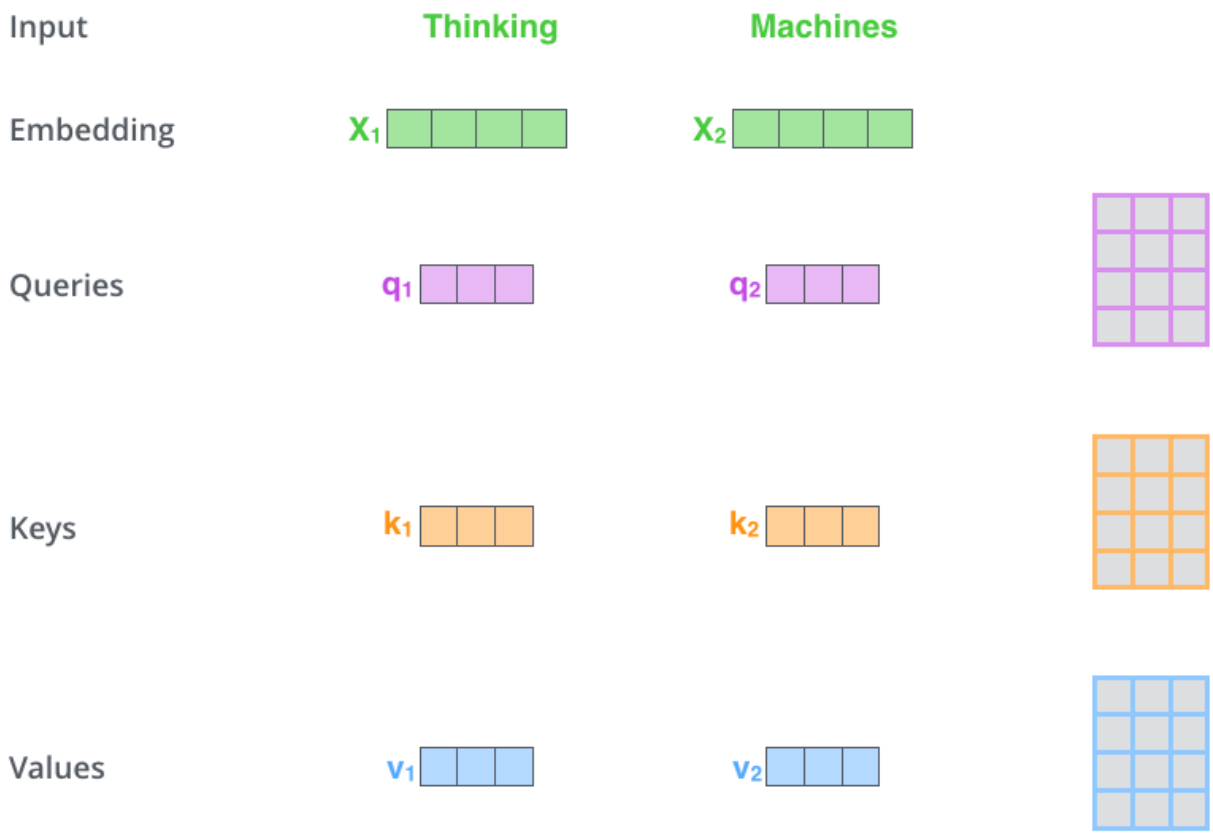


W^K

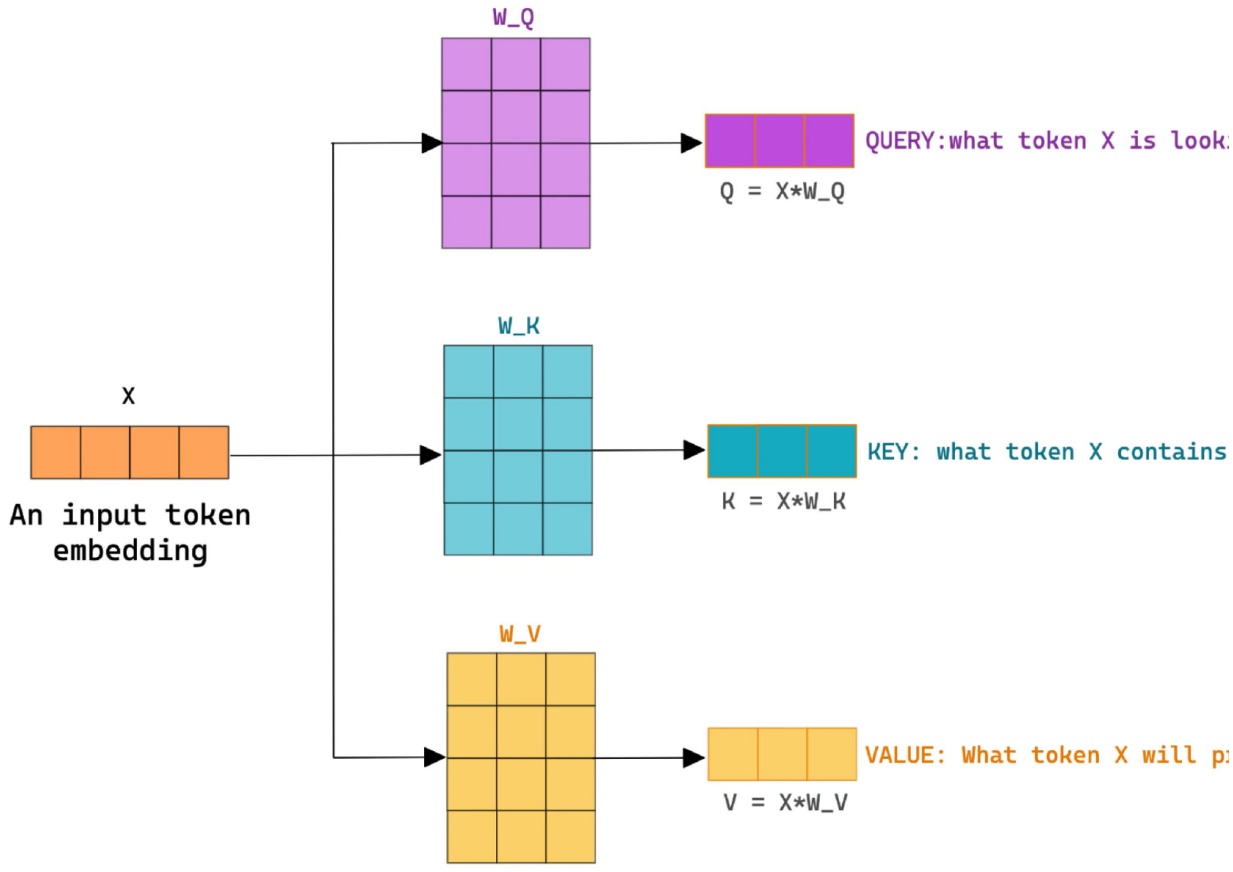


W^V

Każde ze słów jest przemnożone przez te macierze:



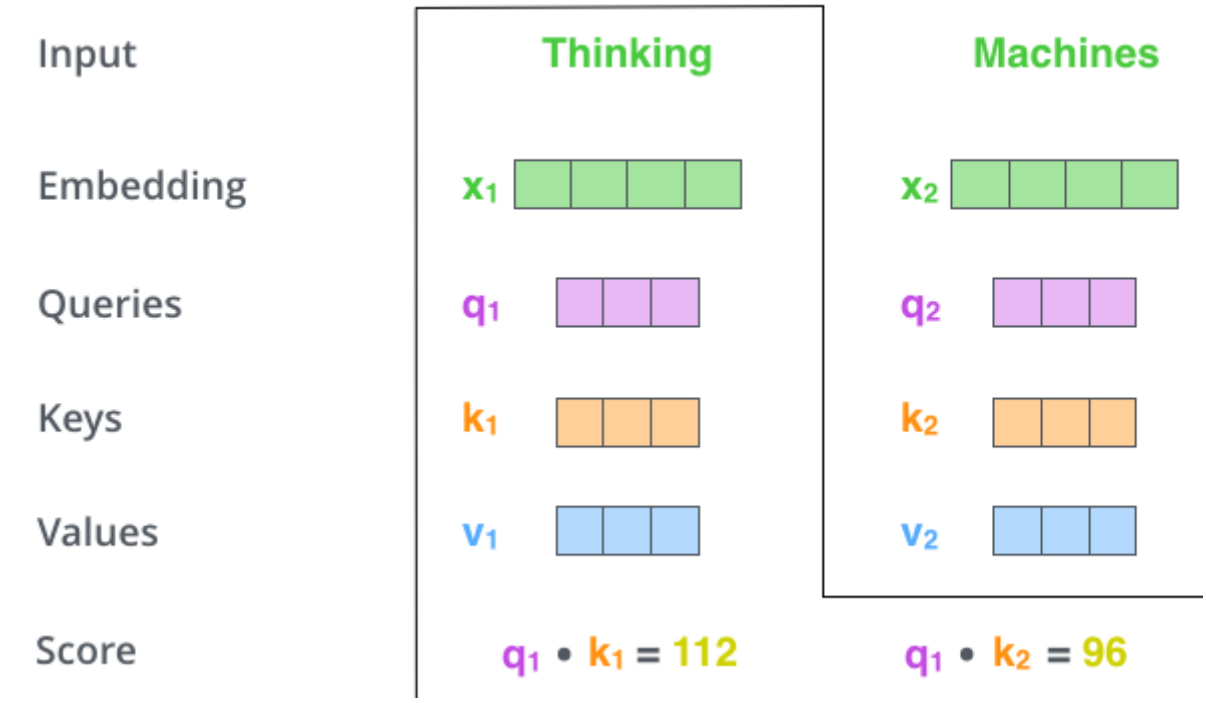
W ten sposób dla każdego słowa (embedingu) otrzymaliśmy wektory query, key i value. Raczej nie tłumaczymy tych słów na język polski.



4 Obliczenie attention - na wektorach

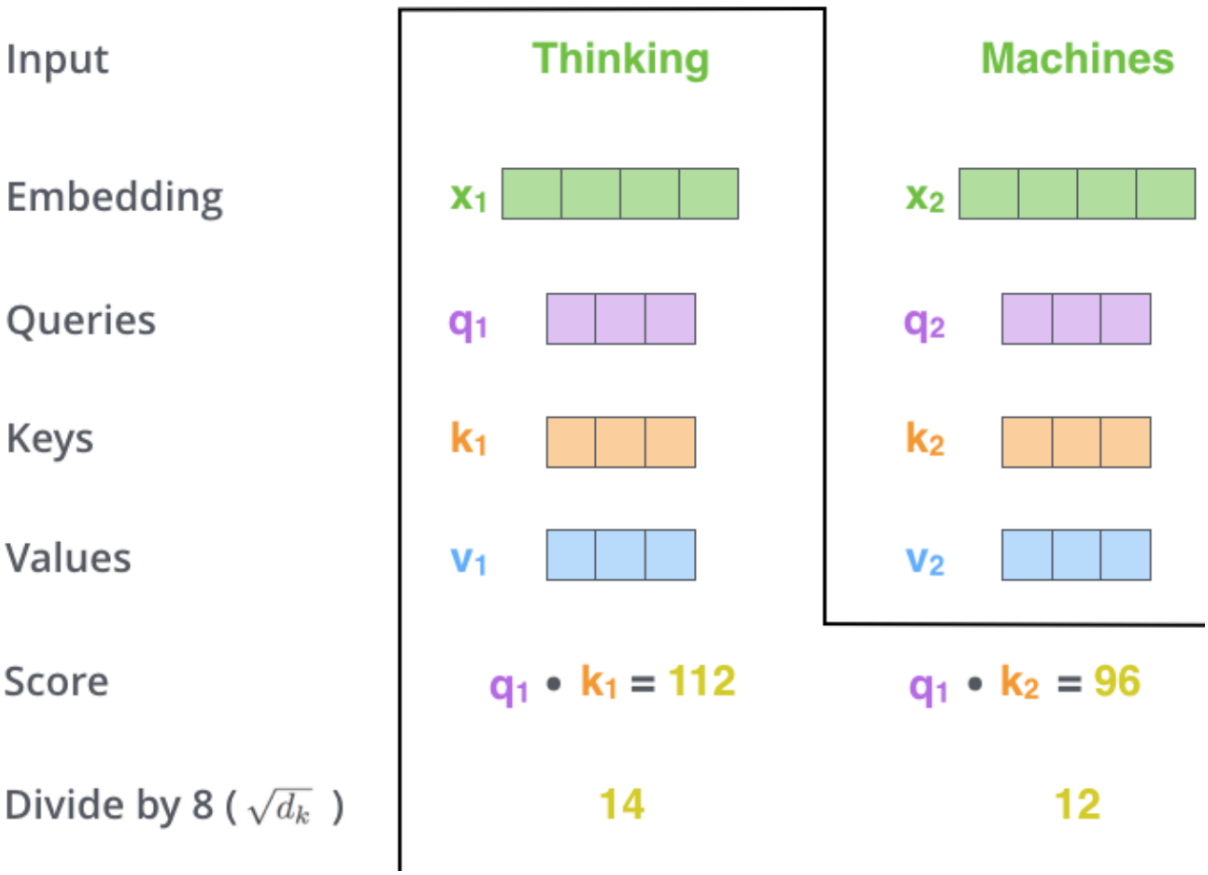
Obliczamy attention dla pierwszego słowa. Interesuje nas wynik jaki uzyskuje nasze z słowo z KAŻDYM słowem występującym.

Przemnażamy query (tego słowa) * key (po koeli każdego występującego słowa):

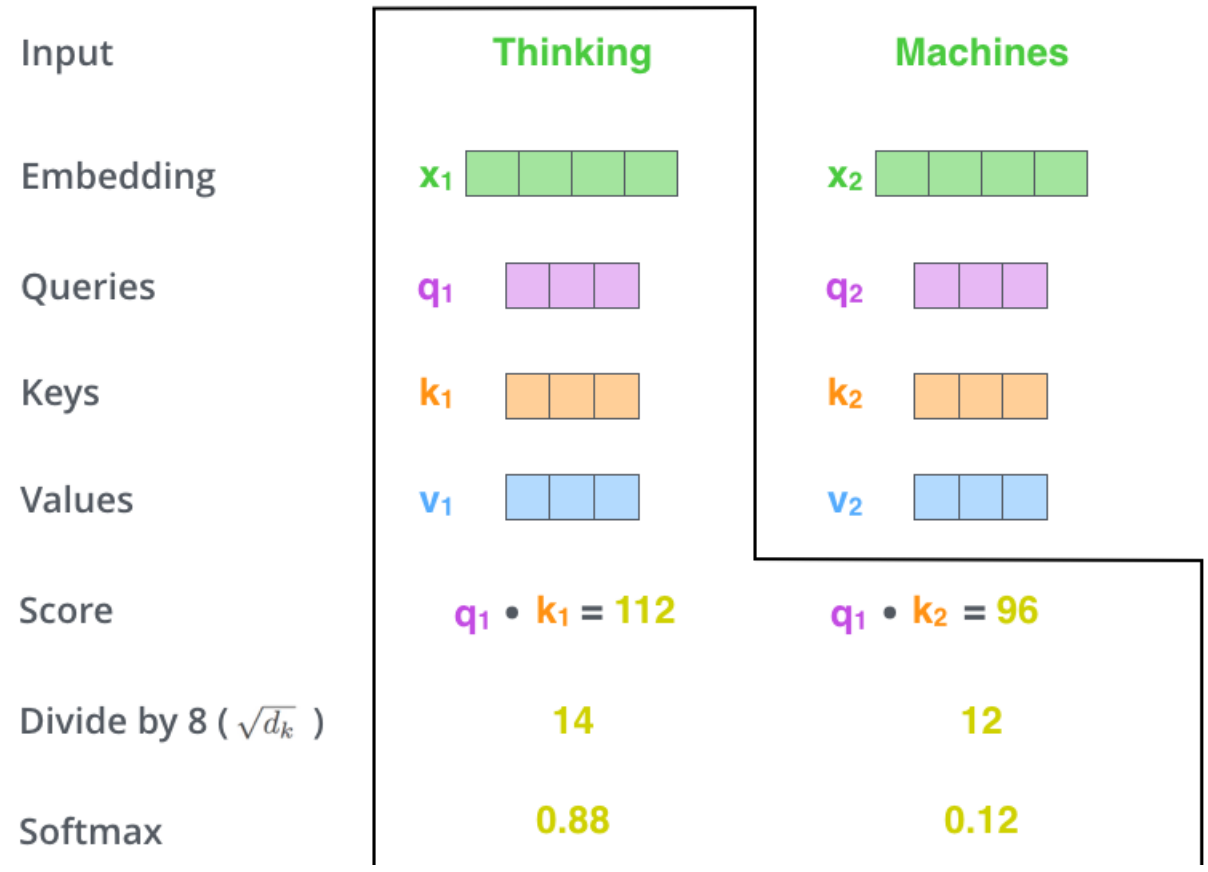


Dzielimy przez pierwiastek z wymiaru wektoru k:

(na podanym przykładzie wzięto wymiar k = 6 4)

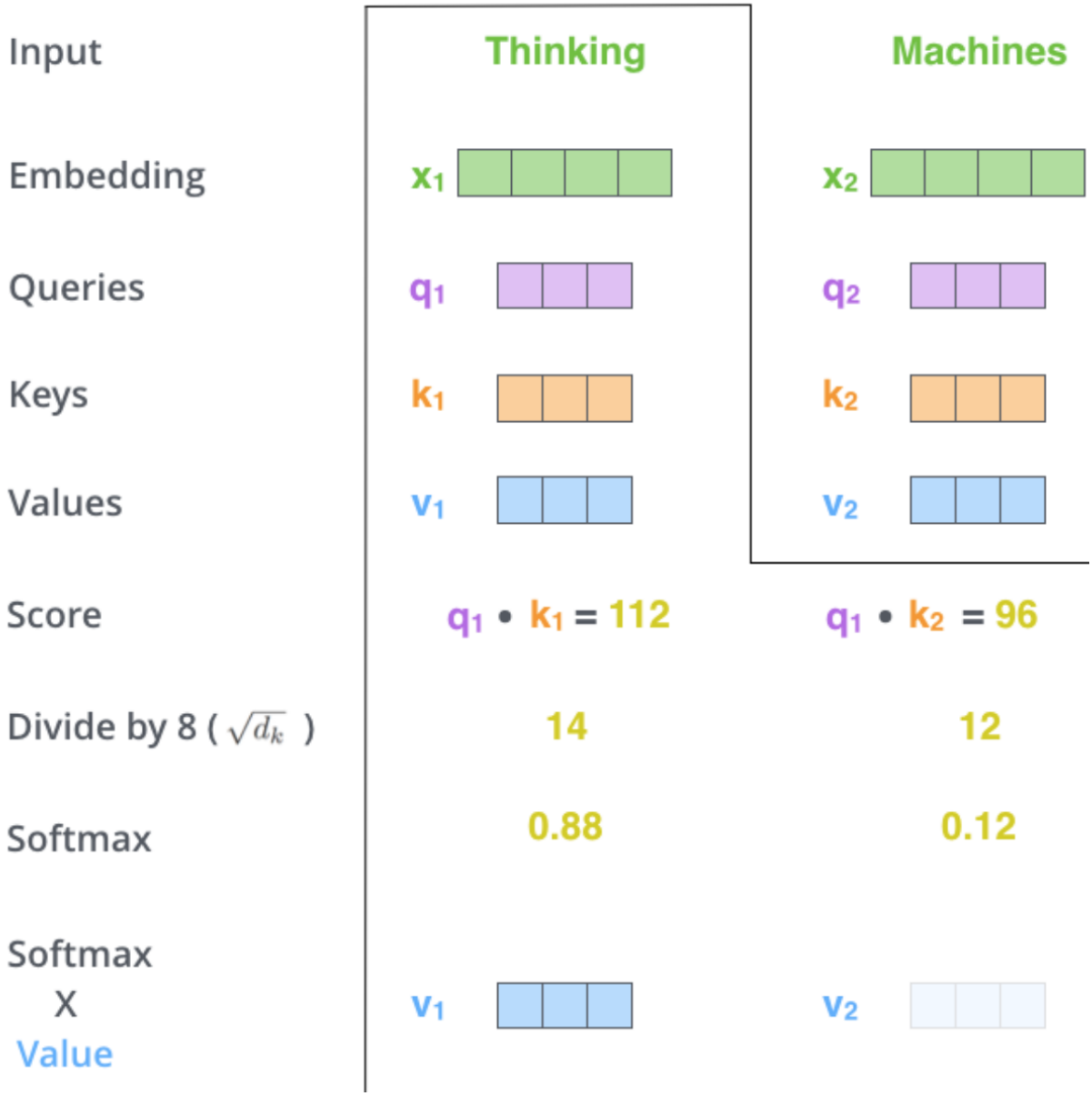


Używamy softmaxa - normalizujemy wartości, żeby wartości były dodatnie i sumowały się do 1.

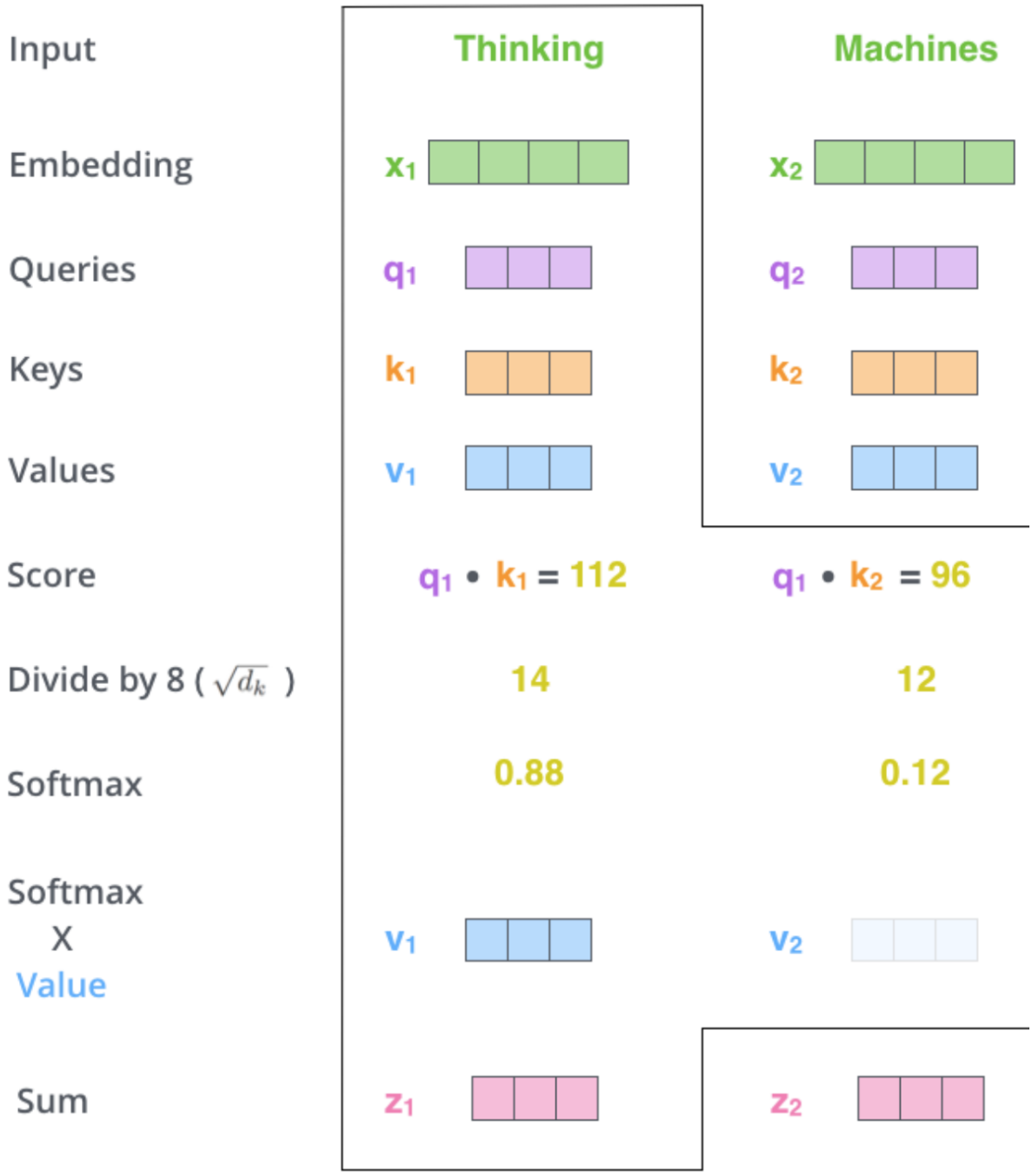


Zauważ, że największy wyniki osiągamy tutaj w liczeniu danego słowa do samego siebie. Jest to naturalne, bo dane słowo reprezentuje siebi
Możemy mieć jednak inne słowa, które będą miały wysoki wynik, co oznacza, że mają do siebie jakiś związek (np bird i później w zdaniu

Teraz wartość naszego softmaxa przemnażamy przez wektor value. Nie chcemy tutaj naruszyć zbytnio naruszyć wartości naszych słów, któ
ważne, ale jednocześnie chcemy wyciszyć te słowa, które nic nam nie wnoszą. (Słowa z softmaxem ~ 0.9 zostaną z dużymi liczbami,
z softmaxem ~ 0.1 będą miały nikły wpływ)



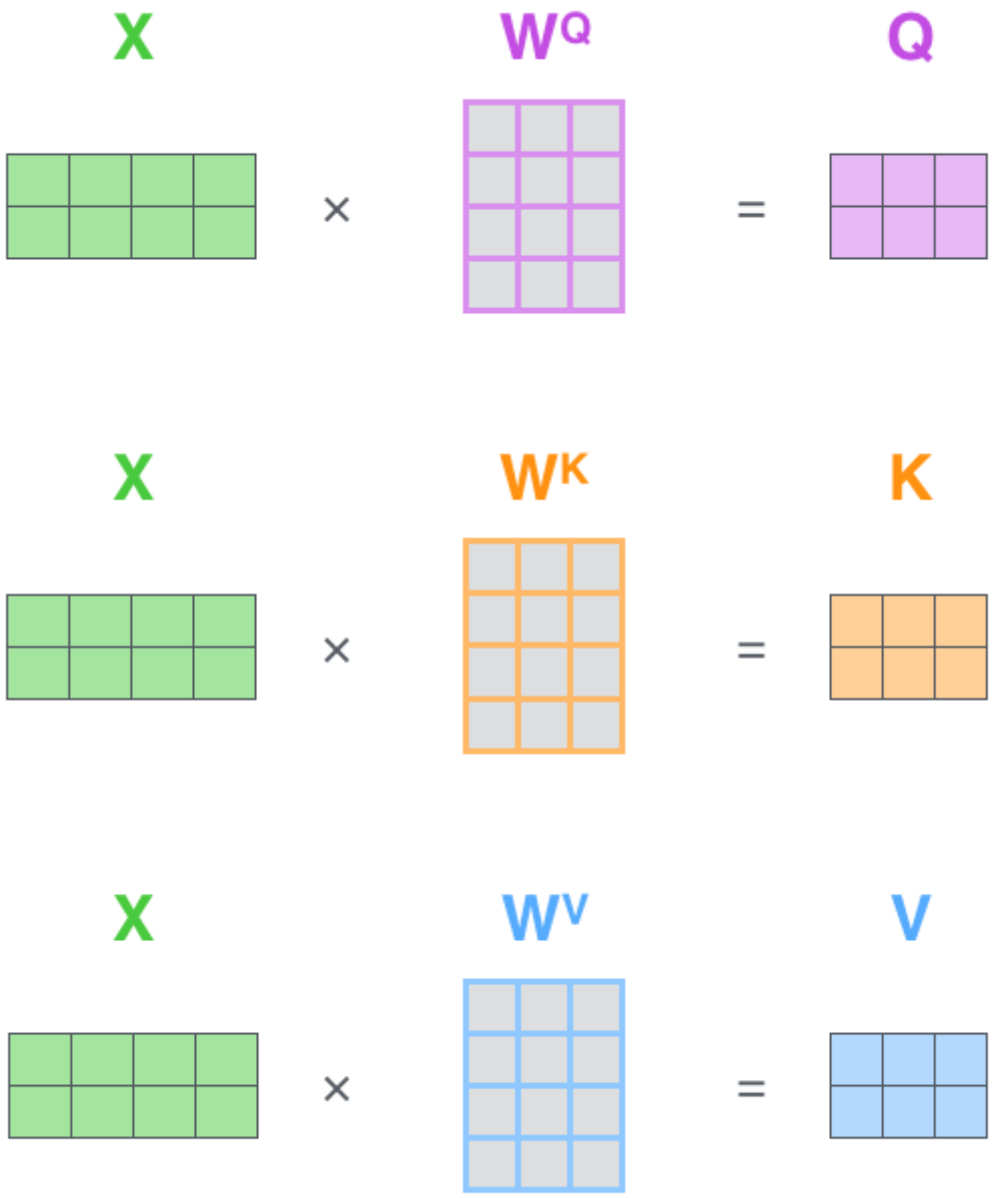
W ten sposób po zebraniu naszego attention w całość, zyskaliśmy nasz nowy wektor, który dalej wyślemy w naszej sieci - do warstwty feed
dalszej architekturze na kolejnych zajęciach)



5 Obliczenie attention - na macierzach

Wektory dobrze wizualizują co siedzieje w attention, ale w rzeczywistym scenariuszu wszystko robimy na macierzach - aby było szybciej.

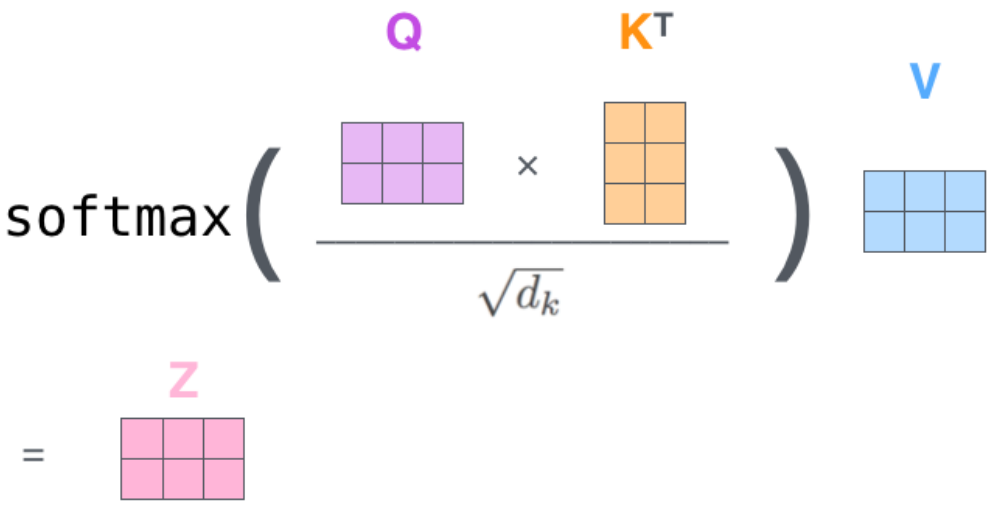
Analogicznie jak z wektorami towrzymy macierze query, key i value:



(każdy wiersz w X odpowiada danemu słowu - to co wcześniej robiliśmy koło siebie, teraz zrobimy "na raz")

W skrócie:

1. Przemnażamy query i key (key jest transponowane, aby macierze się zgadzały)
2. Dzielimy na pierwiastek z wymiaru key (k)
3. Używamy softmaxa
4. Mnożymy razy wartości values
5. Tadam 🐱


Q
K^T
V
softmax
×
$\sqrt{d_k}$

Z
=

In [1]:

```python
import numpy as np
import math

def softmax(x, axis=-1):
    """
    Standardowy softmax (stabilny numerycznie).
    Dla każdej linii (np. dla każdego tokena w sekwencji)
    przelicza wartości na rozkład prawdopodobieństwa.
    """
    x_max = np.max(x, axis=axis, keepdims=True)
    e_x = np.exp(x - x_max)  # stabilizacja numeryczna
    return e_x / np.sum(e_x, axis=axis, keepdims=True)

def scaled_dot_product_attention(Q, K, V, mask=None, verbose=True):
    """
    Implementacja mechanizmu Scaled Dot-Product Attention krok po kroku.

    Parametry:
    -----------
    Q : np.ndarray, shape (..., seq_len_q, dk)
        Zapytania (queries)
    K : np.ndarray, shape (..., seq_len_k, dk)
        Klucze (keys)
    V : np.ndarray, shape (..., seq_len_k, dv)
        Wartości (values)
    mask : np.ndarray lub None
        Maska (np. do maskowania paddingu lub przyszłych tokenów)
    verbose : bool
        Czy drukować pośrednie wyniki

    Zwraca:
    --------
    output : (..., seq_len_q, dv)
        Wynik ważonej sumy wartości
    attn_weights : (..., seq_len_q, seq_len_k)
        Wagi uwagi po softmaxie
    """

    dk = Q.shape[-1]

    if verbose:
        print("==== INPUTS ====")
        print(f"Q shape: {Q.shape}")
        print(f"K shape: {K.shape}")
        print(f"V shape: {V.shape}\n")
        print("Q =", np.round(Q, 3))
        print("K =", np.round(K, 3))
        print("V =", np.round(V, 3))
        print("================\n")

    # Liczymy „scores” – czyli miarę podobieństwa zapytań Q i kluczy K
    scores = np.matmul(Q, K.transpose(0, 2, 1)) if Q.ndim == 3 else np.matmul(Q, K.T)
    if verbose:
        print("Scores (QK^T):")
        print(np.round(scores, 3), "\n")

    # Skalowanie przez sqrt(dk)
    scores = scores / math.sqrt(dk)
    if verbose:
        print(f"Scaled scores (dzielone przez sqrt({dk}) = {math.sqrt(dk):.3f}):")
        print(np.round(scores, 3), "\n")

    # Dodatke: Dodajemy maskę (jeśli jest)
    if mask is not None:
        scores = scores + mask
        if verbose:
            print("After adding mask:")
            print(np.round(scores, 3), "\n")

    # Softmax – zamienia scores na rozkład wag uwagi
    attn_weights = softmax(scores, axis=-1)
    if verbose:
        print("Attention weights po softmax:")
        print(np.round(attn_weights, 3))
        print(f"Każdy wiersz sumuje się do: {np.sum(attn_weights, axis=-1)}\n")

    # Mnożymy przez V, żeby uzyskać ważoną sumę wartości
    output = np.matmul(attn_weights, V)
    if verbose:
        print("Wynik po pomnożeniu przez V:")
        print(np.round(output, 3))
        print("============================\n")

    return output, attn_weights
```

```python
# 🔍 Quick sanity check
np.random.seed(0)
Q = np.random.rand(1, 3, 4)  # batch=1, query len=3, dk=4
K = np.random.rand(1, 5, 4)  # key len=5
V = np.random.rand(1, 5, 6)  # value dim=6

out, w = scaled_dot_product_attention(Q, K, V)

print("FINAL OUTPUT SHAPES:")
print("Output shape:", out.shape)
print("Weights shape:", w.shape)
```

```
==== INPUTS ====
Q shape: (1, 3, 4)
K shape: (1, 5, 4)
V shape: (1, 5, 6)

Q = [[[0.549 0.715 0.603 0.545]
  [0.424 0.646 0.438 0.892]
  [0.964 0.383 0.792 0.529]]]
K = [[[0.568 0.926 0.071 0.087]
  [0.02  0.833 0.778 0.87 ]
  [0.979 0.799 0.461 0.781]
  [0.118 0.64  0.143 0.945]
  [0.522 0.415 0.265 0.774]]]
V = [[[0.456 0.568 0.019 0.618 0.612 0.617]
  [0.944 0.682 0.36  0.437 0.698 0.06 ]
  [0.667 0.671 0.21  0.129 0.315 0.364]
  [0.57  0.439 0.988 0.102 0.209 0.161]
  [0.653 0.253 0.466 0.244 0.159 0.11 ]]]
================

Scores (QK^T):
[[[1.064 1.55  1.812 1.124 1.164]
  [0.947 1.663 1.829 1.369 1.295]
  [1.005 1.415 2.028 0.972 1.281]]]

Scaled scores (dzielone przez sqrt(4) = 2.000):
[[[0.532 0.775 0.906 0.562 0.582]
  [0.474 0.831 0.914 0.684 0.648]
  [0.502 0.707 1.014 0.486 0.64 ]]]

Attention weights po softmax:
[[[0.172 0.219 0.25  0.177 0.181]
  [0.156 0.223 0.242 0.193 0.186]
  [0.166 0.204 0.277 0.163 0.19 ]]]
Każdy wiersz sumuje się do: [[1. 1. 1.]]

Wynik po pomnożeniu przez V:
[[[0.672 0.539 0.394 0.297 0.403 0.259]
  [0.675 0.535 0.411 0.29  0.397 0.249]
  [0.67  0.539 0.385 0.29  0.395 0.263]]]
============================

FINAL OUTPUT SHAPES:
Output shape: (1, 3, 6)
Weights shape: (1, 3, 5)
```

In [2]:

```python
# SELF-ATTENTION DLA JEDNEGO ZDANIA

import numpy as np
import matplotlib.pyplot as plt
import math

# softmax i funkcja attention
def softmax(x, axis=-1):
    x_max = np.max(x, axis=axis, keepdims=True)
    e_x = np.exp(x - x_max)
    return e_x / np.sum(e_x, axis=axis, keepdims=True)

def scaled_dot_product_attention(Q, K, V, mask=None):
    dk = Q.shape[-1]
    scores = np.matmul(Q, K.transpose(0, 2, 1)) / math.sqrt(dk)
    if mask is not None:
        scores = scores + mask
    attn_weights = softmax(scores, axis=-1)
    output = np.matmul(attn_weights, V)
    return output, attn_weights


# Dane wejściowe (symulacja)
np.random.seed(42)

tokens = ["I", "like", "big", "cats"]
seq_len = len(tokens)
d_k = 4
d_v = 6

Q = np.random.rand(1, seq_len, d_k)
K = np.random.rand(1, seq_len, d_k)
V = np.random.rand(1, seq_len, d_v)

print("Q shape:", Q.shape)
print("K shape:", K.shape)
print("V shape:", V.shape)

# Obliczenie uwagi
out, attn_weights = scaled_dot_product_attention(Q, K, V)

print("\nAttention weights shape:", attn_weights.shape)
print("Każdy wiersz sumuje się do 1 →", np.sum(attn_weights, axis=-1))

# Wizualizacja wag uwagi
fig, ax = plt.subplots(figsize=(6, 5))
im = ax.imshow(attn_weights[0], cmap="Blues")
ax.set_xticks(np.arange(seq_len))
ax.set_yticks(np.arange(seq_len))
ax.set_xticklabels(tokens)
ax.set_yticklabels(tokens)
plt.setp(ax.get_xticklabels(), rotation=45, ha="right", rotation_mode="anchor")
ax.set_xlabel("Na które słowa patrzymy (Keys)")
ax.set_ylabel("Które słowo patrzy (Queries)")
ax.set_title("🧠 Self-Attention: 'I like big cats'")

for i in range(seq_len):
    for j in range(seq_len):
        text = ax.text(j, i, f"{attn_weights[0, i, j]:.2f}",
                       ha="center", va="center", color="black", fontsize=9)

plt.colorbar(im, fraction=0.046, pad=0.04)
plt.show()
```

```
Q shape: (1, 4, 4)
K shape: (1, 4, 4)
V shape: (1, 4, 6)

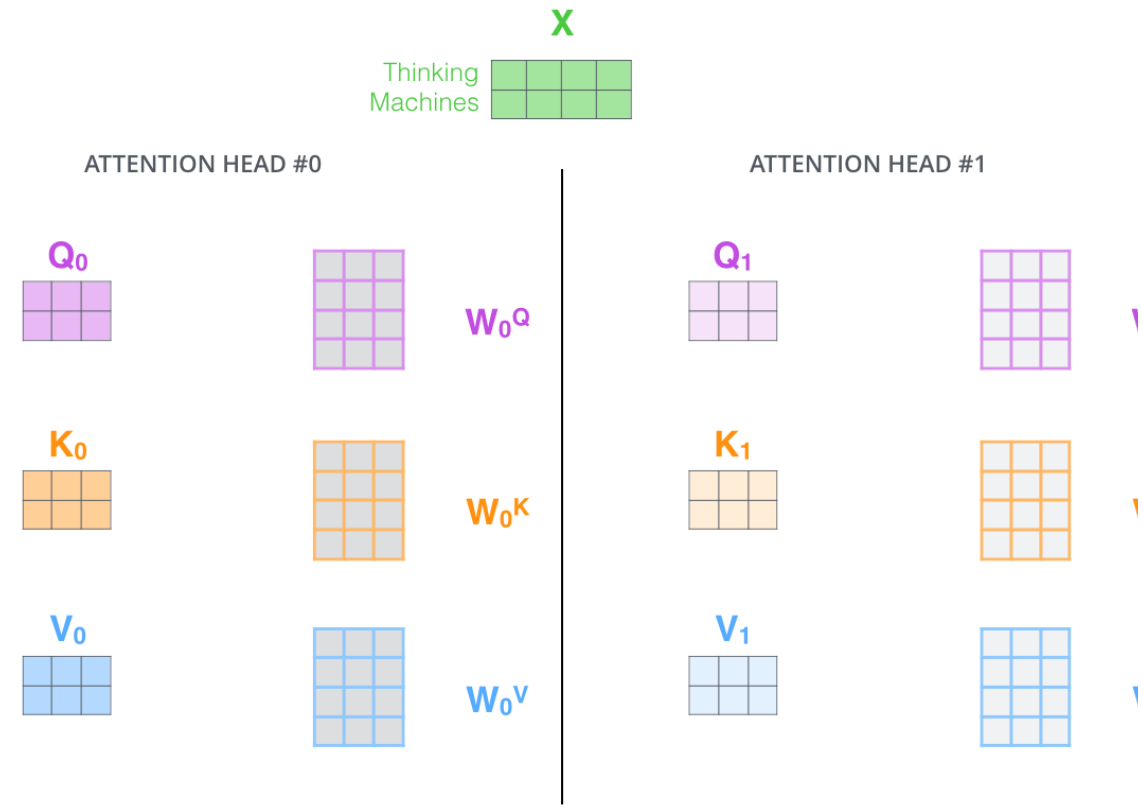
Attention weights shape: (1, 4, 4)
Każdy wiersz sumuje się do 1 → [[1. 1. 1. 1.]]
```

```
/opt/pyvenv/lib/python3.13/site-packages/IPython/core/pylabtools.py:170: UserWarning: Glyph 129504 (\N{BRAIN}) missi
font(s) DejaVu Sans.
  fig.canvas.print_figure(bytes_io, **kw)
```



Multi-Head Attention

Jakby sam mechanizm uwagi był za prosty :) W transformerach mozemy spotkać coś co nazywa się Multi-Head Attention



Jest to rozwinięcie samego attention, która pozwala modelowi uczyć się wielu sposobów "patrzenia" na kontekst słów w zdaniu.

Zamiast jednej atencji (jednego zestawu Q, K, V), tworzymy ich kilka (np. 8), a następnie łączymy wyniki w jedną reprezentację.

In [3]:
```
import torch
import torch.nn as nn
import torch.nn.functional as F

torch.manual_seed(0)
```

Out[3]:
```
<torch._C.Generator at 0x7f7e4f552bf0>
```

1 Wejście

Wyobraźmy sobie zdanie: "The animal didn't cross the street because it was too tired".

Dla uproszczenia stwórzmy losowe wektory (embeddingi) dla 12 słów.

In [4]:
```
seq_len = 12   # liczba słów
d_model = 64  # rozmiar wektora (embedding)
x = torch.rand(seq_len, d_model)
print("Kształt wejścia:", x.shape)
```

```
Kształt wejścia: torch.Size([12, 64])
```

2 Attention (Single-Head)

Przekształca każde słowo (wektor) w nowe reprezentacje używając trzech macierzy wag: W_Q, W_K, W_V (to co wcześniej)

In [5]:
```
d_k = d_model  # dla prostoty
W_Q = nn.Linear(d_model, d_k)
W_K = nn.Linear(d_model, d_k)
W_V = nn.Linear(d_model, d_k)

Q = W_Q(x)
K = W_K(x)
V = W_V(x)

print("Kształty Q, K, V:", Q.shape, K.shape, V.shape)
```

```
Kształty Q, K, V: torch.Size([12, 64]) torch.Size([12, 64]) torch.Size([12, 64])
```

In [6]:
```
scores = torch.matmul(Q, K.T) / torch.sqrt(torch.tensor(d_k, dtype=torch.float32))
attn_weights = F.softmax(scores, dim=-1)
Z = torch.matmul(attn_weights, V)

print("Kształt macierzy wynikowej Z (jedna głowa):", Z.shape)
```

```
Kształt macierzy wynikowej Z (jedna głowa): torch.Size([12, 64])
```

3 Multi head attention

Teraz zamiast jednej atencji, tworzymy kilka "głów" — każda z własnymi W_Q, W_K, W_V. Każda głowa "patrzy" na dane w inny sposób

In [7]:
```
num_heads = 8
head_dim = d_model // num_heads  # 64 rozmiar modelu / 8 głow = 8 rozmiar jednej głowy

W_Q = nn.Linear(d_model, d_model)
W_K = nn.Linear(d_model, d_model)
W_V = nn.Linear(d_model, d_model)

Q = W_Q(x).view(seq_len, num_heads, head_dim)
K = W_K(x).view(seq_len, num_heads, head_dim)
V = W_V(x).view(seq_len, num_heads, head_dim)

print("Kształt Q po podziale na głowy:", Q.shape)  # [12, 8, 8]
```

```
Kształt Q po podziale na głowy: torch.Size([12, 8, 8])
```

Obliczamy uwagę osobno dla każdej głowy

In [8]:
```
Z_heads = []
for h in range(num_heads):
    q, k, v = Q[:, h, :], K[:, h, :], V[:, h, :]
    scores = torch.matmul(q, k.T) / torch.sqrt(torch.tensor(head_dim, dtype=torch.float32))
    attn = F.softmax(scores, dim=-1)
    z = torch.matmul(attn, v)
    Z_heads.append(z)

print("Z_heads:", [h.shape for h in Z_heads], "Len Z_heads:", len(Z_heads))

# [12, 64] po połączeniu wszystkich głów - to co mieliśmy przed
Z_multi = torch.cat(Z_heads, dim=-1)
print("Z po połączeniu wszystkich głów:", Z_multi.shape)
```

```
Z_heads: [torch.Size([12, 8]), torch.Size([12, 8]), torch.Size([12, 8]), torch.Size([12, 8]), torch.Size([12, 8]),
torch.Size([12, 8]), torch.Size([12, 8]), torch.Size([12, 8])] Len Z_heads: 8
Z po połączeniu wszystkich głów: torch.Size([12, 64])
```

Każda głowa:

- z_i.shape = [12 , 8] → czyli 12 tokenów × 8 cech (bo head_dim = 8).

Mamy 8 głów * 8 cech = 64 → To jest surowe połączenie wyników głów — wszystkie perspektywy „obok siebie”.

Warstwa łączenia (Output Projection) - Po złączeniu wszystkich głów, mnożymy przez macierz WO, żeby wrócić do wymiaru d_model. To dane do kolejnych warstw transformera.

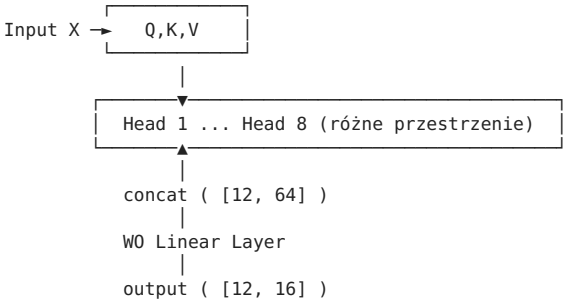
In [9]:
```
W_O = nn.Linear(d_model, d_model)
output = W_O(Z_multi)
print("Ostateczny kształt po WO:", output.shape)
```

```
Ostateczny kształt po WO: torch.Size([12, 64])
```

Dlaczego nie kończymy na łączeniu wszystkich głów?

Bo bez W_O:

- każda głowa miałaby „swoje" cechy, ale nie byłoby interakcji między nimi, sieć dalej miałaby tensor [1 2 , 6 4] — niezgodny z o
  d_model.
- W_O gwarantuje, że każdy token po multi-head attention ma z powrotem wektor o tym samym rozmiarze, co na wejściu (d_model) –
  architektura Transformera pozostaje spójna w każdej warstwie.



Chwila, chwila.. przecież mieliśmy wymiar [1 2 , 6 4] po złączeniu głów i po warstwie łaczenia, która ma wrócić nas do wymiaru d
dokładnie ten sam wymiar?! Co to za oszustwa 😠

Jeśli w modelu:

In [10]:
```
d_model == num_heads * head_dim
```

Out[10]:
```
True
```

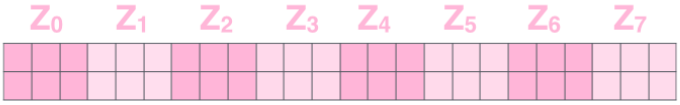
to:

- po konkatenacji (concat) masz [seq_len, 6 4]
- po W_O dalej masz [seq_len, 6 4]

Więc — tak, wymiar liczbowo się nie zmienia.

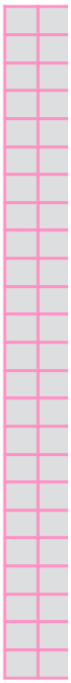
W_O nie istnieje po to, żeby zmienić rozmiar tensora. Istnieje po to, żeby nauczyć się mieszać informacje z różnych głów.

1) Concatenate all the attention heads



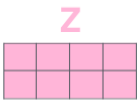
2) Multiply with a weight matrix W^O that was trained jointly with the model

X



3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN

=



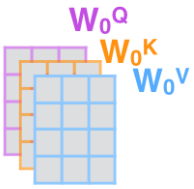
1) This is our input sentence*

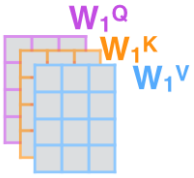
Thinking Machines

2) We embed each word*

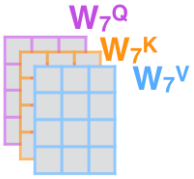


3) Split into 8 heads. We multiply X or R with weight matrices

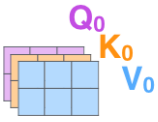


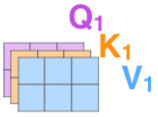


...

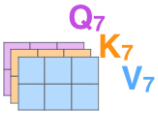


4) Calculate attention using the resulting Q/K/V matrices





...



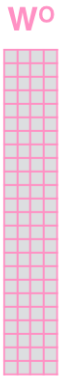
5) Concatenate the resulting then multiply with weight m produce the output of the la





...

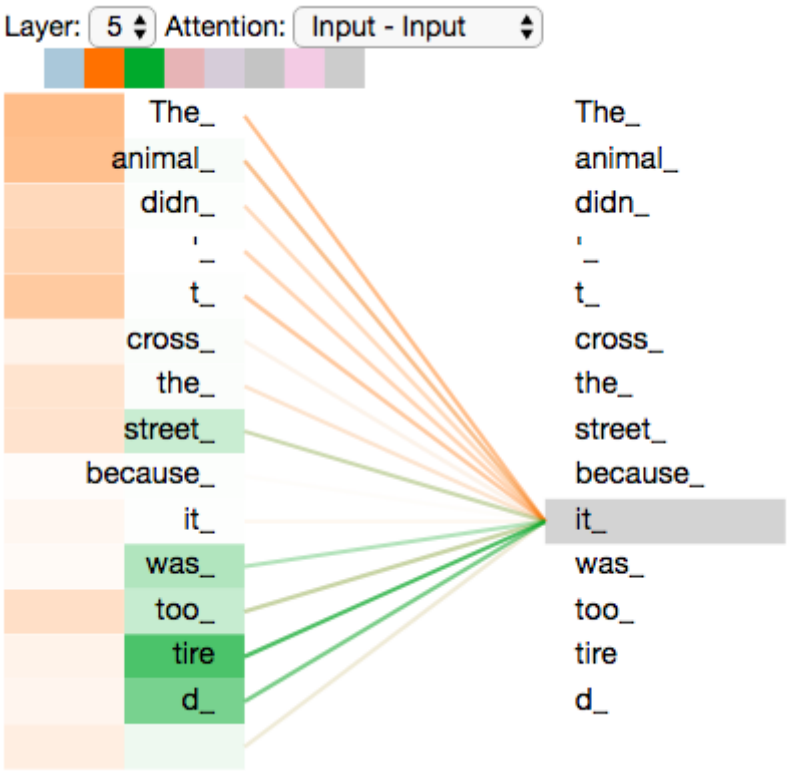




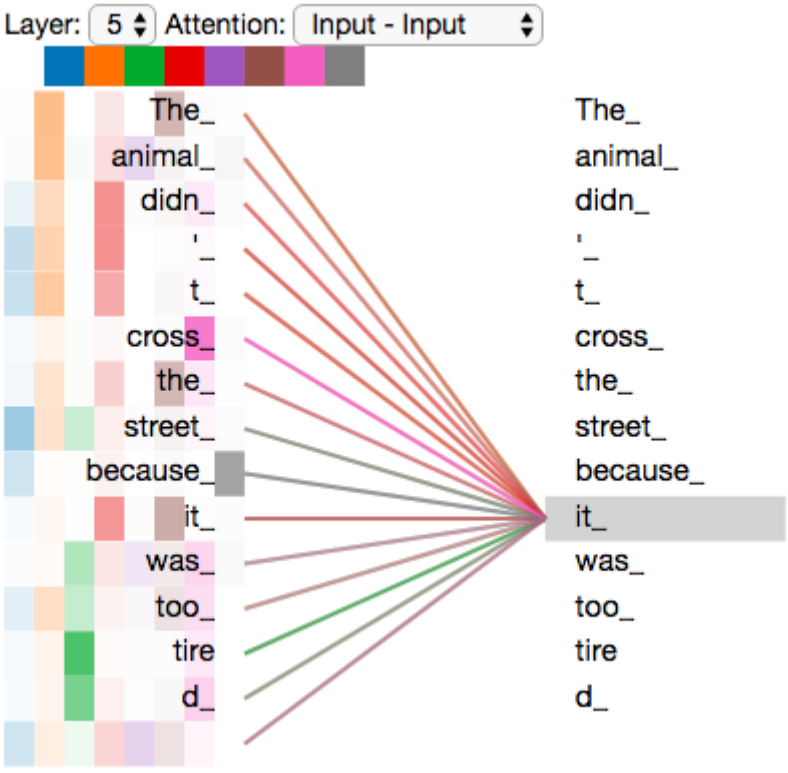
* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



Teraz możemy porównać jak przykładowe dwie głowy traktują nasze zdanie wyjściowe:



Ale po dodaniu 8 głów:



Zadania

Zadanie 1

Liczymy attention:

1. Utwórz wektory:

- Q = [1, 0]
- K1 = [1, 1]
- K2 = [0, 1]
- V1 = [2, 0]
- V2 = [0, 2]

1. Oblicz podobieństwo - score 1 i score 2 : Q * K^T

2. Skalujemy - scaled 1 i scaled 2 : dzielenie przez sqrt(d_k), uznajmy że d_k = 2

3. zastosuj softmax - probs

(Podpowiedź: To już nie będzie pojedynczy numerek a wektor: `scaled_scores = np.array([scaled1, scaled2])` i na ty softmax)

1. Suma ważona Values - output

2. Odpowiedz na pytanie zawarte w print - które słowo (Key) miało największy wpływ na wynik?

In [11]:
```
import numpy as np

# 0) Dane
Q = np.array([1., 0.])
K1 = np.array([1., 1.])
K2 = np.array([0., 1.])
V1 = np.array([2., 0.])
V2 = np.array([0., 2.])


# 1) podobieństwa
score1 = np.dot(Q, K1)
score2 = np.dot(Q, K2)

# 2) scaled scores
d_k = 2
scaled1 = score1 / np.sqrt(d_k)
scaled2 = score2 / np.sqrt(d_k)

# 3) softmax
def softmax(x):
    exp_x = np.exp(x - np.max(x))
    return exp_x / np.sum(exp_x)

scaled_scores = np.array([scaled1, scaled2])
probs = softmax(scaled_scores)

# 4) weighted sum of V
output = probs[0] * V1 + probs[1] * V2

# Print zadanie 1
print("Scores:", score1, score2)
print("Scaled scores:", scaled_scores)
print("Softmax probabilities:", probs)
print("Output attention (wektor):", output)
print("Interpretacja: K1 ma największy wpływ na wynik")
```

```
Scores: 1.0 0.0
Scaled scores: [0.70710678 0.        ]
Softmax probabilities: [0.66976155 0.33023845]
Output attention (wektor): [1.3395231 0.6604769]
Interpretacja: K1 ma największy wpływ na wynik
```

Zadanie 2

Utwórz prostą implementację multi-head attention dla dwóch głów, korzystając z z Zadania 1.

Każda głowa będzie miała własne wektory Qi, Ki, Vi, maciesze Wq, Wk i Wv oraz własne wyniki softmax i output.

Na końcu połączysz wyniki obu głów (konkatenacja), aby uzyskać końcowy wektor.

Zwizualizuj wagi uwagi - forma dowolna

In [12]:

```python
import numpy as np
import matplotlib.pyplot as plt

def softmax(x):
    x = x - np.max(x, axis=-1, keepdims=True)
    exp_x = np.exp(x)
    return exp_x / np.sum(exp_x, axis=-1, keepdims=True)

def scaled_dot_product_attention(Q, K, V):
    d_k = K.shape[-1]

    scores = Q @ K.T / np.sqrt(d_k)
    weights = softmax(scores)
    output = weights @ V

    return output, weights

# Dane bazowe
K = np.array([[1., 1.], [0., 1.]])
V = np.array([[2., 0.], [0., 2.]])
Q = np.array([[1., 0.], [0.5, 1.0]])

d_model = 2
d_head = 2

# Głowa 1
Wq1 = np.array([[1., 0.], [0., 1.]])
Wk1 = np.array([[1., 0.], [0., 1.]])
Wv1 = np.array([[1., 0.], [0., 1.]])

# Głowa 2
Wq2 = np.array([[0.5, 0.2], [0.1, 0.7]])
Wk2 = np.array([[0.8, 0.1], [0.3, 0.9]])
Wv2 = np.array([[0.6, 0.4], [0.2, 0.8]])

Wq_heads = [Wq1, Wq2]
Wk_heads = [Wk1, Wk2]
Wv_heads = [Wv1, Wv2]

head_outputs = []
head_weights = []

for i in range(2):
    # Własne projekcje dla każdej głowy
    Qi = Q[i:i+1] @ Wq_heads[i]
    Ki = K @ Wk_heads[i]
    Vi = V @ Wv_heads[i]

    output, weights = scaled_dot_product_attention(
        Qi, Ki, Vi
    )

    head_outputs.append(output)
    head_weights.append(weights)

    print(f"Głowa {i+1}:")
    print("Wagi:", weights)
    print("Output:", output)

# Konkatenacja wyników obu głów
final_output = np.concatenate(head_outputs, axis=-1)
print("\nWynik końcowy:", final_output)

# Wizualizacja wag
fig, axes = plt.subplots(1, 2, figsize=(8, 3))

for i, ax in enumerate(axes):
    ax.imshow(head_weights[i], cmap="Blues", vmin=0, vmax=1)
    ax.set_title(f"Głowa {i+1}")
    ax.set_xticks([0, 1], labels=["K1", "K2"])
    ax.set_yticks([0], labels=["Q"])

    for j, weight in enumerate(head_weights[i][0]):
        ax.text(j, 0, f"{weight:.3f}",
                ha="center", va="center", color="black")

plt.tight_layout()
plt.show()
```

```
Głowa 1:
Wagi: [[0.66976155 0.33023845]]
Output: [[1.3395231 0.6604769]]
Głowa 2:
Wagi: [[0.56329817 0.43670183]]
Output: [[0.85063854 1.14936146]]

Wynik końcowy: [[1.3395231  0.6604769  0.85063854 1.14936146]]
```

